

Design Patterns Used in the LibraryApp Project

Prepared on: 2026-04-02

Project scanned: - client/ - Flutter mobile/web client - server/ - FastAPI backend - admin/ - Next.js admin portal

Scope notes: - I focused on real source files and architecture files. - I did not treat generated/build/vendor content as architectural evidence, such as .g.dart, .dart_tool, .next, node_modules, build, and platform-generated files.

1. Executive Summary

This project does not use only one design pattern. It uses a combination of patterns across the client, server, and admin applications.

The strongest and most consistent patterns in this codebase are:

1. MVVM / ViewModel pattern in the Flutter client
2. Repository pattern in the Flutter client
3. Observer / reactive state pattern through Riverpod and React Context
4. Single source of truth / centralized state store
5. Dependency Injection
6. Application Factory pattern in the FastAPI backend
7. Router / Controller pattern for APIs
8. ORM + DTO / Data Mapper style separation on the backend
9. Transaction Script pattern in the admin API routes
10. Factory and Singleton-like infrastructure helpers

There are also a few supporting architectural patterns:

- Layered architecture
- Feature-based modular architecture
- Middleware pipeline
- Layout composition
- Facade / wrapper around external APIs

2. Architecture Overview

At a high level, your system is split like this:

Flutter Client

Views -> ViewModels -> Repositories -> Services -> Backend APIs

FastAPI Server

App Factory -> Routers -> Dependencies -> Models / Schemas -> Database

Next.js Admin

Layouts / Components -> Route Handlers -> DB Helpers / Auth Helpers

This structure already shows that the project is layered and modular, not a flat file-by-file app.

3. Flutter Client Patterns

3.1 MVVM / ViewModel Pattern

What it is

MVVM means:

- View handles rendering and UI events
- ViewModel handles state and UI logic
- Model holds the data objects

Why your code matches this

Your Flutter features are clearly separated into:

- views/
- viewmodels/
- models/

The View listens to state and triggers actions on the ViewModel. The ViewModel loads data, updates state, and handles errors.

Code evidence

File: client/lib/features/home/views/home_view.dart

```
class HomeView extends ConsumerWidget {
  const HomeView({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final homeState = ref.watch(homeViewModelProvider);

    return Scaffold(
      backgroundColor: Pallete.scaffoldBackground,
      body: SafeArea(
        child: homeState.isLoading
          ? _buildLoading()
          : homeState.error != null
            ? _buildError(homeState.error!, ref)
            : _buildContent(context, ref, homeState),
      ),
    );
  }
}
```

File: client/lib/features/home/viewmodels/home_viewmodel.dart

```
@riverpod
class HomeViewModel extends _$HomeViewModel {
  @override
  HomeState build() {
    Future.microtask(() => _loadInitialData());
    return const HomeState(isLoading: true);
  }

  Future<void> _loadInitialData() async {
    if (!ref.mounted) return;
    await Future.wait([_loadBooks(), _loadCategories()]);
  }

  Future<void> _loadBooks() async {
    if (!ref.mounted) return;
    try {
      final repository = ref.read(bookRepositoryProvider);
      final result = await repository.getAllBooks();
      if (!ref.mounted) return;
      result.fold(
        (failure) =>
          state = state.copyWith(isLoading: false, error: failure.message),
        (books) => state = state.copyWith(
          books: books,
          isLoading: state.categories.isEmpty,
        ),
      );
    } catch (e) {
      if (!ref.mounted) return;
      state = state.copyWith(isLoading: false, error: e.toString());
    }
  }
}
```

Clear conclusion

This is a real MVVM-style implementation, not just folder naming. The View depends on the ViewModel state, and the ViewModel coordinates repository calls and state updates.

3.2 Repository Pattern

What it is

The Repository pattern hides data access details behind a clean API so the UI or ViewModel does not directly call HTTP or SQL.

Why your code matches this

Your ViewModels call repository providers such as:

- bookRepositoryProvider
- loanRepositoryProvider
- favoritesRepositoryProvider
- userRepositoryProvider

These repositories encapsulate backend calls and data conversion.

Code evidence

File: client/lib/data/repository/book_repository.dart

@riverpod

```
BookRepository bookRepository(Ref ref) {  
  return BookRepository();  
}
```

```
class BookRepository {  
  Future<Either<Failure, List<Book>>> getAllBooks() async {  
    try {  
      final res = await AuthenticatedHttpClient.get(  
        Uri.parse('${ServerConstant.serverURL}/books'),  
      );  
  
      if (res.statusCode == 200) {  
        final List<dynamic> data = jsonDecode(res.body);  
        final books = data.map((e) {  
          return Book(  
            id: e['id']?.toString() ?? '',  
            title: e['title']?.toString() ?? '',  
            author: e['author']?.toString() ?? 'Unknown Author',  
            category: e['category']?.toString() ?? '',  
            description:  
              e['description']?.toString() ?? 'No description available.',  
            rating: (e['rating'] as num?)?.toDouble() ?? 0.0,  
            publicationYear: e['publication_year'] as int? ?? 0,  
            copiesOwned: e['copies_owned'] as int? ?? 0,  
            image: e['image']?.toString() ??  
              'https://via.placeholder.com/150',  
            language: e['language']?.toString() ?? 'English',  
            pages: e['pages'] as int? ?? 200,  
            ratingCount: e['rating_count'] as int? ?? 0,  
          );  
        }).toList();  
        return right(books);  
      } else {  
        return left(Failure(res.body));  
      }  
    } catch (e) {  
      return left(Failure(e.toString()));  
    }  
  }  
}
```

```
}  
}
```

Clear conclusion

Yes, your Flutter app is using the Repository pattern consistently.

3.3 Observer / Reactive State Pattern

What it is

The Observer pattern means parts of the UI subscribe to state changes, and when the state changes, the UI updates automatically.

Why your code matches this

Riverpod is a reactive state system. Your widgets use `ref.watch(...)`, and some viewmodels use `ref.listen(...)`. That is classic observer-style behavior.

Code evidence

File: `client/lib/features/profile/viewmodels/profile_viewmodel.dart`

```
@riverpod  
class ProfileViewModel extends _$ProfileViewModel {  
  @override  
  ProfileState build() {  
    ref.listen(asgardeoAuthProvider, (previous, next) {  
      final prevUserId = previous?.user?.sub;  
      final nextUserId = next.user?.sub;  
      if (previous?.isLoggedIn != next.isLoggedIn || prevUserId !=  
nextUserId) {  
        Future.microtask(loadProfileData);  
      }  
    });  
  
    ref.listen(currentUserProvider, (previous, next) {  
      if (previous?.id != next?.id) {  
        Future.microtask(loadProfileData);  
      }  
    });  
  
    Future.microtask(() => loadProfileData());  
    return const ProfileState(isLoading: true);  
  }  
}
```

File: `client/lib/features/home/views/home_view.dart`

```
final homeState = ref.watch(homeViewModelProvider);
```

Clear conclusion

This is a strong example of the Observer pattern implemented through Riverpod.

3.4 Single Source of Truth / Centralized Store Pattern

What it is

This pattern means one central state object owns a domain's current data, instead of many screens keeping their own copies.

Why your code matches this

Your global notifiers keep shared state for:

- favorites
- loans and reservations
- current user
- direct auth state

Screens and viewmodels read from the same central store.

Code evidence

File: client/lib/core/providers/favorites_notifier.dart

```
class FavoritesState {
  final List<Book> favorites;
  final Set<String> favoriteIds;
  final bool isLoading;
  final String? error;
  final String memberId;

  const FavoritesState({
    this.favorites = const [],
    this.favoriteIds = const {},
    this.isLoading = false,
    this.error,
    this.memberId = '',
  });
}

@Riverpod(keepAlive: true)
class FavoritesNotifier extends _$FavoritesNotifier {
  @override
  FavoritesState build() {
    Future.microtask(() => loadFavorites());
    return const FavoritesState(isLoading: true);
  }
}
```

```

void setMemberId(String memberId) {
  final normalizedMemberId = memberId.trim();
  if (state.memberId != normalizedMemberId) {
    state = state.copyWith(memberId: normalizedMemberId);
    loadFavorites();
  }
}
}
}

```

Clear conclusion

This is more than just state management. It is a central store pattern where one provider is the source of truth for a domain.

3.5 Facade / Wrapper Pattern

What it is

A Facade provides a simpler interface in front of a more complex subsystem.

Why your code matches this

AuthenticatedHttpClient hides token-reading and header-building details so repositories do not repeat that logic.

Code evidence

File: client/lib/core/services/authenticated_http_client.dart

```

class AuthenticatedHttpClient {
  static const FlutterSecureStorage _secureStorage = FlutterSecureStorage();
  static const String _accessTokenKey = 'asgardeo_access_token';

  static Future<String?> _getToken() async {
    return _secureStorage.read(key: _accessTokenKey);
  }

  static Future<Map<String, String>> _authHeaders([
    Map<String, String>? extra,
  ]) async {
    final token = await _getToken();
    final headers = <String, String>{};
    if (token != null && token.isNotEmpty) {
      headers['Authorization'] = 'Bearer $token';
    }
    if (extra != null) {
      headers.addAll(extra);
    }
    return headers;
  }
}

```

```

static Future<http.Response> get(
    Uri url, {
    Map<String, String>? headers,
}) async {
    final h = await _authHeaders(headers);
    return http.get(url, headers: h);
}
}

```

Clear conclusion

This is a small but valid Facade / wrapper pattern around authenticated HTTP behavior.

3.6 Feature-Based Modular Architecture

What it is

Feature-based architecture organizes files by business feature instead of only by technical type.

Why your code matches this

Your client is organized into:

- features/home
- features/search
- features/profile
- features/borrowed
- features/book_details
- features/wishlist

Each feature contains its own views, widgets, and often its own viewmodel.

Example structure

```

client/lib/features/
  home/
    views/
    viewmodels/
    widgets/
  search/
    views/
    viewmodels/
    widgets/
  profile/
    views/
    viewmodels/
    widgets/

```


Clear conclusion

This is a modular feature-first architecture, which works well together with MVVM.

4. FastAPI Server Patterns

4.1 Application Factory Pattern

What it is

The Application Factory pattern creates the app through a function instead of configuring everything inline in a single global file.

Why your code matches this

Your backend app is created through `create_app()` and returned after routers, middleware, rate limiting, CORS, static assets, and security headers are configured.

Code evidence

File: `server/app/main.py`

```
from .app_factory import create_app
```

```
app = create_app()
```

File: `server/app/app_factory.py`

```
def create_app() -> FastAPI:
    app = FastAPI(title="Library App API", lifespan=lifespan)

    default_limit = os.getenv("DEFAULT_RATE_LIMIT", "60/minute")
    limiter = create_limiter(default_limits=[default_limit])
    app.state.limiter = limiter

    app.add_middleware(
        CORSMiddleware,
        allow_origins=allowed_origins,
        allow_credentials=allow_credentials,
        allow_methods=["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS",
"HEAD"],
        allow_headers=["*"],
    )

    app.include_router(general.router)
    app.include_router(auth.router)
    app.include_router(books.router)
    app.include_router(loans.router)
    app.include_router(category.router)
    app.include_router(favorites.router)
```

```

app.include_router(users.router)
app.include_router(reservations.router)
app.include_router(settings.router)

return app

```

Clear conclusion

Yes, your backend clearly uses the Application Factory pattern.

4.2 Dependency Injection Pattern

What it is

Dependency Injection provides external dependencies to functions and classes instead of hardcoding them directly.

Why your code matches this

FastAPI injects:

- database sessions
- token verification dependencies

using `Depends(...)`.

Code evidence

File: `server/app/dependencies.py`

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

File: `server/app/routers/books.py`

```

router = APIRouter(
    prefix="/books", tags=["books"],
    dependencies=[Depends(verify_access_token)]
)

@router.get("", response_model=List[book_schema.Book])
def get_books(db: Session = Depends(get_db)):
    books = db.query(book_model.Book).all()
    return [_book_to_response(b) for b in books]

```

Clear conclusion

This is a textbook use of Dependency Injection in FastAPI.

4.3 Router / Controller Pattern

What it is

The Router / Controller pattern groups related endpoints by domain responsibility.

Why your code matches this

Your backend has separate route modules for:

- auth
- books
- loans
- category
- favorites
- users
- reservations
- settings

Each router owns one domain area.

Code evidence

File: server/app/app_factory.py

```
app.include_router(general.router)
app.include_router(auth.router)
app.include_router(books.router)
app.include_router(loans.router)
app.include_router(category.router)
app.include_router(favorites.router)
app.include_router(users.router)
app.include_router(reservations.router)
app.include_router(settings.router)
```

File: server/app/routers/books.py

```
router = APIRouter(
    prefix="/books", tags=["books"],
    dependencies=[Depends(verify_access_token)]
)
```

Clear conclusion

Yes, your backend is clearly structured using router/controller modules.

4.4 ORM + DTO / Data Mapper Style Separation

What it is

This pattern separates:

- database objects
- API transport schemas
- mapping logic

Why your code matches this

You do not expose SQLAlchemy models directly. You define:

- SQLAlchemy models in `models/`
- Pydantic schemas in `pydantic_schemas/`
- explicit conversion logic in routers

Code evidence

File: `server/app/models/book.py`

```
class Book(Base):
    __tablename__ = "books"

    id = Column(TEXT, primary_key=True)
    title = Column(TEXT)
    author_id = Column(TEXT, ForeignKey("authors.id"))
    category_id = Column(TEXT, ForeignKey("categories.id"))

    category_rel = relationship("Category", back_populates="books")
    author_rel = relationship("Author", back_populates="books")

    @property
    def category(self):
        return self.category_rel.name if self.category_rel else None
```

File: `server/app/pydantic_schemas/book.py`

```
class BookBase(BaseModel):
    title: str
    author: str
    category: str
    description: str
    rating: float
    publication_year: int
    copies_owned: int
    image: str

class Book(BookBase):
    id: str

    model_config = ConfigDict(from_attributes=True)
```

File: server/app/routers/books.py

```
def _book_to_response(book_obj: book_model.Book) -> book_schema.Book:
    return book_schema.Book(
        id=str(book_obj.id),
        title=str(book_obj.title or ""),
        author=str(book_obj.author or ""),
        category=str(book_obj.category or ""),
        description=str(book_obj.description or ""),
        rating=(
            float(cast(float, book_obj.rating)) if book_obj.rating is not None
        else 0.0
    ),
        publication_year=(
            int(cast(int, book_obj.publication_year))
            if book_obj.publication_year is not None
            else 0
        ),
        copies_owned=(
            int(cast(int, book_obj.copies_owned))
            if book_obj.copies_owned is not None
            else 0
        ),
        image=str(book_obj.image or ""),
        language=str(book_obj.language or "English"),
        pages=(int(cast(int, book_obj.pages)) if book_obj.pages is not None
        else 200),
        rating_count=(
            int(cast(int, book_obj.rating_count))
            if book_obj.rating_count is not None
            else 0
        ),
    )
```

Clear conclusion

This is a clean model/schema/mapping separation. It is very close to a Data Mapper plus DTO approach.

4.5 Middleware Pipeline Pattern

What it is

Middleware creates a request-processing pipeline. Each middleware layer can inspect, reject, modify, or pass along the request/response.

Why your code matches this

Your backend has:

- CORS middleware

- max body size middleware
- security header middleware

These form a request pipeline before the route handler runs.

Code evidence

File: server/app/app_factory.py

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=allowed_origins,
    allow_credentials=allow_credentials,
    allow_methods=["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS",
"HEAD"],
    allow_headers=["*"],
)

app.add_middleware(
    MaxRequestBodySizeMiddleware, max_body_size=max_request_size_bytes
)

@app.middleware("http")
async def security_headers_middleware(request: Request, call_next):
    content_length = request.headers.get("content-length")
    if content_length and content_length.isdigit():
        if int(content_length) > max_request_size_bytes:
            return JSONResponse(
                status_code=413,
                content={"detail": "Request body too large"},
            )

    response = await call_next(request)
    response.headers.setdefault("X-Content-Type-Options", "nosniff")
    response.headers.setdefault("X-Frame-Options", "DENY")
    response.headers.setdefault("Referrer-Policy", "no-referrer")
    return response
```

Clear conclusion

Yes, the backend uses a middleware pipeline pattern.

4.6 Application Lifecycle / Lifespan Hook Pattern

What it is

This pattern centralizes startup and shutdown work.

Why your code matches this

Your backend uses a lifespan context manager to initialize database metadata on startup.

Code evidence

File: server/app/startup.py

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    yield
```

Clear conclusion

This is a valid application lifecycle pattern, often used together with an application factory.

5. Next.js Admin Patterns

5.1 React Context + Provider Pattern

What it is

React Context provides shared state and behavior to the component tree without prop-drilling.

Why your code matches this

Your admin app uses an AuthContext and exposes it through an AuthProvider. That provider is applied globally in app/providers.tsx.

Code evidence

File: admin/lib/auth/auth-context.tsx

```
const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const checkSession = async () => {
      try {
        const res = await fetch("/api/auth/me", { credentials: "include" });
        if (res.ok) {
          const data = await res.json();
          if (data.authenticated && data.user) {
```

```

        setUser(data.user);
        setIsAuthenticated(true);
    }
} finally {
    setIsLoading(false);
}
};

checkSession();
}, []);

return (
    <AuthContext.Provider
        value={{
            user,
            isAuthenticated,
            isLoading,
            login,
            logout,
        }}
    >
        {children}
    </AuthContext.Provider>
);
}

File: admin/app/providers.tsx
export function Providers({ children }: { children: ReactNode }) {
    return <AuthProvider>{children}</AuthProvider>;
}

```

Clear conclusion

Yes, the admin app uses the Context + Provider pattern for authentication state.

5.2 Layout Composition Pattern

What it is

Layout composition means the app builds pages by composing shared layout shells and reusable UI blocks.

Why your code matches this

The dashboard layout wraps all dashboard pages with a sidebar shell. This is a clean composition pattern.

Code evidence

```
File: admin/app/(dashboard)/layout.tsx
export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="min-h-screen bg-gray-50">
      <Sidebar />
      <main className="min-h-screen lg:pl-64">{children}</main>
    </div>
  );
}
```

Clear conclusion

The admin UI uses component composition and shared layout shells in a clear way.

5.3 Transaction Script Pattern

What it is

The Transaction Script pattern puts request logic directly inside route handlers instead of moving it into a deep service/domain layer.

Why your code matches this

Your admin route handlers often:

1. read request parameters
2. run SQL
3. validate input
4. transform the result
5. return JSON

all inside one route file.

Code evidence

```
File: admin/app/api/dashboard/route.ts
export async function GET() {
  try {
    await ensureFineInfrastructure();

    const usersResult = await queryOne<{ count: string }>(
      "SELECT COUNT(*) as count FROM users",

```

```

);
const activeUsers = parseInt(usersResult?.count || "0");

const inventoryResult = await queryOne<{ total: string }>(
  "SELECT COALESCE(SUM(copies_owned), 0) as total FROM books",
);
const totalInventory = parseInt(inventoryResult?.total || "0");

const finesResult = await queryOne<{ total: string; count: string }>(
  `SELECT
    COALESCE(SUM(fine_amount), 0) as total,
    COUNT(*) as count
  FROM fines
  WHERE LOWER(COALESCE(status, 'unpaid')) = 'unpaid'`,
);

return NextResponse.json({
  stats: {
    activeUsers,
    totalInventory,
  },
});
} catch (error) {
  return NextResponse.json(
    { error: "Internal server error" },
    { status: 500 },
  );
}
}

```

File: admin/app/api/books/route.ts

```

export async function POST(request: NextRequest) {
  const auth = await verifyAdmin(request);
  if (auth.error) return auth.error;

  try {
    const body = await request.json();

    if (
      !body.title ||
      typeof body.title !== "string" ||
      body.title.trim().length < 1 ||
      body.title.trim().length > 500
    ) {
      return NextResponse.json(
        { error: "Title is required and must be 1-500 characters" },
        { status: 400 },
      );
    }

    const id = `b${Math.floor(100000 + Math.random() * 900000)}`;
    const columns = await getBookColumnSet();
    const usesAuthorId = columns.has("author_id") && !columns.has("author");
  }
}

```

Clear conclusion

The admin API is mostly using Transaction Script, not a large service-layer architecture.

5.4 Shared Gateway / Singleton-like Resource Pattern

What it is

A shared gateway or singleton-like resource pattern centralizes access to a reusable infrastructure object such as a DB pool.

Why your code matches this

Your Postgres pool is created once at module level and reused through helper functions like `getClient`, `query`, and `queryOne`.

Code evidence

File: `admin/lib/database/connection.ts`

```
const { config: poolConfig, diagnostics: dbDiagnostics } =
  resolvePoolConfig();

const pool = new Pool({
  ...poolConfig,
  max: 20,
  min: 2,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 5000,
});

export async function getClient(): Promise<PoolClient> {
  try {
    return await pool.connect();
  } catch (error) {
    logDbOperationError("connect", error);
    throw error;
  }
}

export async function query<T>(text: string, params?: unknown[]): Promise<T[]>
{
  let client: PoolClient | null = null;
  try {
    client = await getClient();
    const result = await client.query(text, params);
    return result.rows as T[];
  } finally {
    client?.release();
  }
}
```

```
}
```

Clear conclusion

This behaves like a singleton/shared gateway for database access.

5.5 Simple Factory Pattern

What it is

A factory creates configured objects so callers do not need to know construction details.

Why your code matches this

Your storage client is created through a helper function that takes config and returns a fully configured S3Client.

Code evidence

File: admin/lib/storage/client.ts

```
export function createStorageClient(config: StorageConfig): S3Client {  
  return new S3Client({  
    region: config.region,  
    endpoint: config.s3Endpoint,  
    credentials: {  
      accessKeyId: config.accessKeyId,  
      secretAccessKey: config.secretAccessKey,  
    },  
    forcePathStyle: true,  
  });  
}
```

Clear conclusion

This is a simple and valid factory pattern.

6. Cross-Cutting Patterns

6.1 Layered Architecture

Why it is present

Across the repo, your code is separated into layers:

- UI layer

- state / orchestration layer
- data access layer
- service / infrastructure layer

Example

```
client:
  views -> viewmodels -> repositories -> services

server:
  app_factory -> routers -> dependencies -> models/schemas -> db

admin:
  layouts/components -> route handlers -> db/auth/storage helpers
```

Clear conclusion

This project is strongly layered even though each application uses different frameworks.

6.2 Feature-Based Modularity

Why it is present

The client is grouped by feature, and the backend/admin are grouped by domain.

Examples:

- home, search, profile, borrowed, wishlist
- books, users, loans, favorites, settings

Clear conclusion

This is good modular decomposition and helps maintainability.

7. Patterns That Are Partial or Lightly Used

These patterns are present, but not in a heavy textbook way:

Facade

- Present in client service wrappers such as `AuthenticatedHttpClient`
- Present in some helper modules that simplify infrastructure access

Singleton

- Not used as a formal GoF Singleton class
- But effectively present as shared module-level instances like the Postgres pool

Composition

- Strongly present in UI layout building
- Especially in Flutter widgets and Next.js layouts/components

Lifecycle hooks

- Present in FastAPI lifespan startup logic
- Also present in React `useEffect` session hydration

8. Patterns That I Do Not Clearly See

I do **not** see strong evidence of these as formal project-wide patterns:

- Builder pattern
- Abstract Factory
- Strategy pattern as interchangeable classes/objects
- Command pattern
- Decorator pattern as explicit wrappers over behavior objects
- State pattern as explicit polymorphic state classes
- Adapter pattern as a major architectural style

Some logic may look similar in small places, but not strongly enough to call them established patterns in this codebase.

9. Final Pattern List

Below is the clean final list of design patterns used in your project.

Strongly used

1. MVVM / ViewModel pattern
2. Repository pattern
3. Observer / reactive state pattern
4. Single source of truth / centralized store
5. Dependency Injection
6. Application Factory pattern
7. Router / Controller pattern
8. ORM + DTO / Data Mapper style separation
9. Transaction Script pattern
10. Layered architecture
11. Feature-based modular architecture

Also present

12. Facade / wrapper pattern
13. Middleware pipeline pattern
14. Application lifecycle / lifespan hook pattern

15. React Context + Provider pattern
16. Layout composition pattern
17. Shared gateway / singleton-like resource pattern
18. Simple Factory pattern

10. Final Verdict

Your project is architecturally stronger than a basic CRUD app.

The client is the most pattern-rich part of the system, especially because it combines:

- MVVM
- repositories
- Riverpod reactive state
- centralized notifiers

The backend is cleanly structured around:

- application factory
- routers
- dependency injection
- schema/model separation
- middleware

The admin app is lighter architecturally, but it still clearly uses:

- React Context Provider
- shared layout composition
- transaction-script route handlers
- shared DB gateway
- simple client factories

If I had to summarize your project in one sentence:

This codebase mainly uses a layered, feature-based architecture with MVVM and Repository on the Flutter side, and Factory + Router + Dependency Injection on the backend side.